ChatGPT

2018–2026 量子操控指令与编译验证、噪声/漂移自适应容错编译、容错权衡与低延迟测控架构的研究蓝图与小课题组可行路线

执行摘要

本报告围绕四条主线形成一张“从语言/IR → 编译/验证 → 噪声与漂移闭环优化 → 容错与系统权衡 → 百纳秒级反馈测控”的近五年研究蓝图，并在“小课题组、设备有限”的约束下，优先推荐可用仿真与云端硬件完成、且具备 **CCF-A（如 POPL/PLDI/ASPLOS/MICRO/ISCA/OSDI/SOSP/ICSE）** 投稿潜力的方向。

关键结论

量子软件栈正在从“仅门级电路”走向“可描述实时控制与反馈”的指令体系：OpenQASM 3 专门面向实时域（real-time domain），将控制流、时序、脉冲与校准纳入同一语言框架，从语言层就承认“量子—经典协同与延迟预算”的重要性。[1] 这意味着：**形式化语义与可验证编译**不再只针对电路等价，还必须覆盖“时序/资源/反馈正确性”。

噪声与漂移并非“次要扰动”，而是编译与执行策略的核心输入：经典工作（如 Noise-Adaptive Compiler Mappings）已经展示利用动态校准数据可显著提升成功率，并明确讨论“时空变化的硬件参数”应被编译利用。[2] 同时，随机化编译（randomized compiling）等方法专门针对系统误差/失配与漂移，将相干误差转化为更可预测的随机噪声，并给出硬件实验验证。[3] 这为“小组无硬件”提供了可行切入：**用公开校准数据+仿真器+少量云端实验**形成闭环评估。

低延迟反馈已进入“可工程化、可开源复现”的阶段：从早期 FPGA 反馈触发延迟约 110 ns（用于实时判决与反馈）[4] 到新近针对动态超导量子计算的读出电子学在 FPGA 上实现 **40 ns** 级读出处理延迟指标[5]，再到开源 RFSoC 控制器（QICK）与开源 FPGA 控制栈（QubiC）形成软硬件全栈参考设计。[6] 说明：即便你们实验设备不足，也可以通过 **“测控-编译协同的数字孪生/仿真平台”** 做出可发表的系统研究，并把硬件验证降到“云端/合作/少量板卡”。

最可行且具备 CCF-A 潜力的推荐课题

推荐课题按“设备需求低 + 可在 18–36 个月产出 + 有明确可验证贡献点”排序（推荐 3 个，满足你希望的 1–3 个优先选题）：

课题 A（最推荐，偏 POPL/PLDI/ICSE）：面向 OpenQASM 3 / OpenPulse 的“时序—反馈语义”形式化与可验证编译（Verified Real-time Quantum Control Compilation）
核心抓手是 OpenQASM 3 将电路、控制流、时序、脉冲和校准统一描述；当前规范详尽但缺少“机器可检验语义+端到端可验证编译链”。[7] 你可以复用并扩展已有量子形式化框架（QWIRE/SQIR/VOQC、CertiQ、VyZX）[8]，把验证对象从“电路等价”推进到“**时序约束、寄存器/测量、反馈分支**”层面。

课题 B（强推荐，偏 ASPLOS/MICRO/ISCA）：利用云端后端校准数据的漂移感知编译 + 随机化编译/动态退耦联合优化（Drift-aware Adaptive Compilation）
以“动态校准数据+噪声模型+编译搜索/学习”为主，硬件仅需云端少量实验。经典 noise-adaptive mapping 已验证利用校准数据可显著提升成功率[2]；随机化编译直接针对失配与漂移[9]；而 Qiskit 侧也提供可插拔的

动态退耦（PadDynamicalDecoupling）编译通道作为强基线。[10] 你们小组可用“公开校准数据日志 + 仿真器 + 周期性云端跑分”建立可复现实验体系。

课题 C（推荐，偏 ASPLOS/MICRO/OSDI）：动态电路与 QEC/反馈工作负载的端到端延迟预算与系统—编译协同（Latency Budget & Co-design）
OpenQASM 3 明确面向 real-time 量子—经典交互[11]，IBM 动态电路实践强调中途测量与经典操作可能拉长执行时间并抵消收益、因此降低 mid-circuit measurement 等延迟是重点改进方向。[12] 同时，QEC 领域对“解码速度/延迟”已出现极快实现（例如 FPGA 分布式 UF 解码在特定噪声模型下报告 ns–几十 ns 级解码时间指标）[13]，以及“低延迟 QEC 硬件实验”开始出现。[14] 你可以用 Stim/PyMatching 等工具建立“QEC+反馈延迟+编译策略”联合模型与仿真平台，在无硬件条件下仍能做出系统级结论。[15]

研究范围与未指定约束的假设

本报告覆盖范围

时间范围以 **2018–2026（截至 2026-03-06）**；主题拆成两组子方向以匹配你的结构要求：

子方向 **3.1–3.3**：围绕“量子操控指令与编译形式化验证”“噪声与漂移自适应容错编译”“容错策略与系统性能权衡”。
子方向 **4.1–4.3**：围绕“可扩展、低延迟、软硬件协同测控架构（含百纳秒级反馈）”以及其与语言/IR、动态电路、实时解码的连接。

你未指定但对路线影响最大的约束项

你没有指定经费、可用硬件、团队技能结构、投稿时间窗。为给出可执行路线，本报告采用“小课题组常见情形”的**工作假设**（后续你可按实际替换参数）：

经费：默认“可覆盖云端量子机少量作业费用/学生算力/可能 1 块开发板或服务器升级”，但无法承担整套超导测控链（AWG、IQ 混频、低噪声放大、稀释制冷等）。
硬件：默认没有专用量子测控机柜；可用若干台 GPU/CPU 服务器或工作站；可访问云端 IBM Quantum 等平台（用于门级或部分动态电路能力）。IBM 动态电路在 2022 年起在部分系统上线，且以 OpenQASM 3 描述为重要接口。[16]
成员能力：你有哈密顿模拟背景与量子计算入门经验；默认你能快速上手 Python/Qiskit/仿真器，并能学习一套形式化工具（Coq/Lean/SMT）或系统原型开发。
时间窗：按 18–36 个月规划，目标在 18–24 个月形成可投 CCF-A 的主稿（或一篇主稿+一篇领域期刊/workshop）。

研究现状综述

本节按子方向 **3.1–3.3、4.1–4.3** 梳理代表性论文/综述、关键方法、开源工具与平台，并在末尾给出比较表（关注假设、实验规模、延迟/鲁棒性指标、是否需要专用硬件）。

子方向 3.1：量子操控指令与编译形式化验证

趋势概括：量子编译器从“正确性靠测试+等价检查”逐步走向“证明助手/SMT 的可验证编译”。典型路线有两类：其一是以 Coq/定理证明为核心、对 IR/优化进行语义证明（QWIRE/SQIR/VOQC）；其二是以“契约+SMT 生成验证条件”的工程化量子编译验证（CertiQ）。[17]

代表性工作与要点（2018–2024）：

QWIRE Practice（2018）：把量子电路语言嵌入 Coq，给出线性类型检查与指称语义（将电路解释为密度矩阵上的超算子），用于证明小型量子程序性质。[18]
VOQC（2019 arXiv / 2021 POPL 版本）：提出 SQIR 作为低层量子 IR，使用 Coq 证明一系列优化变换语义保持，并报告其优化效果与工业编译器可比。[19]
Proving Quantum Programs Correct（ITP 2021）：在 SQIR 框架下进一步验证 Grover、QPE 等算法，强调“IR 设计使证明更容易”的经验教训。[20]
CertiQ（2019）：面向真实 Qiskit 编译器 pass 的“契约式验证框架”，通过注释生成验证条件并交由 SMT 求解；文中报告重实现并验证了大量 Qiskit pass，并发现 Qiskit 实现中的 bug。[21]
VyZX（2023–2024）：面向 ZX-calculus 的图形化量子语言验证库，解决“纸面图推导 vs 证明助手图结构”的鸿沟，并可证明重写规则的正确性。[22]

与“操控指令/脉冲”相关的语言与 IR 背景（为后续研究铺垫）：

OpenQASM 3：扩展到可表达实时控制流、时序、脉冲与校准块，目标聚焦 real-time 域。[23]
OpenPulse：作为 OpenQASM 生态中的校准语法（defcalgrammar "openpulse"），核心抽象是 ports/frames/waveforms，面向硬件控制抽象。[24]
Qiskit Pulse：提出“云端可编程脉冲级指令”，在 IBM 云端系统上校准实现高保真门，并强调可用于最优控制、动态退耦、误差缓解等高级控制。[25]
QIR（Quantum Intermediate Representation）：以 LLVM IR 为承载的量子 IR 规范与生态（QIR Alliance、qir-spec、QAT 等），旨在实现语言与后端互操作并支持混合量子—经典程序。[26]

子方向 3.2：面向噪声与漂移的自适应容错编译机制

趋势概括：从静态映射/优化走向“噪声感知、时间变化感知、乃至在线/闭环自适应”。对小组而言，关键机会是：很多硬件平台公开（或可通过云 API 访问）校准数据，使“无自有硬件也能做系统研究”。

代表性工作与方法谱系：

Noise-Adaptive Compiler Mappings（ASPLOS 2019）：系统化讨论 NISQ 硬件的可靠性与参数时变性，提出多种映射/路由策略对比，并用真实系统测量/校准数据评估，报告相对基线编译器的显著成功率提升。[27]
t|ket〉（2020）：强调跨硬件、面向 NISQ 的高性能编译平台，包含优化与路由并宣称在基准上优于多种编译器；开放源码可用于对比基线。[28]
随机化编译（Randomized compiling，2020 arXiv / 2021 PhysRevX）：针对系统失配、漂移与串扰导致的相干误差，提出通过随机化将其转化为随机噪声并提高可预测性，给出超导平台实验验证。[9]
脉冲级鲁棒控制与云端闭环：Q-CTRL 等工作展示通过云端“脉冲级接口”（Qiskit Pulse）设计对去相位噪声、控制幅度不稳定、串扰等更鲁棒的门操作，并报告提升校准窗口、降低随时间/跨比特的误差波动等指标，直接触及“漂移”问题。[29]
动态退耦（DD）编译插入：Qiskit 提供 PadDynamicalDecoupling 等 pass，在已调度的物理电路中识别 idle 时隙并插入 DD 序列以抑制退相干。[10]

“容错编译”在此子方向的解释：若你现在缺少大规模纠错硬件，建议把“容错”分成两层推进：第一层是 **NISQ 级噪声抑制/误差定制（noise tailoring）与漂移自适应**；第二层才是 **FT 级（表面码/晶格手术）资源与实时反馈约束下的编译**，并用仿真/资源估算替代真实硬件。

子方向 3.3：容错策略与系统性能权衡分析

趋势概括：容错不是单一算法问题，而是跨层权衡：编码距离、循环周期（QEC cycle time）、解码延迟、门集与魔态工厂吞吐、通信/并行度，都会改变“可执行规模与总时间”。近年出现两条对小组友好的路线：一是“资源估算与编译一体化”；二是“解码并行化与延迟建模”。

代表性工作：

“Game of Surface Codes”（2019）：讨论从小规模到大规模表面码计算的策略，强调如何在不同规模下最小化开销（典型的空间—时间权衡讨论入口）。[30]
PRXQuantum 2020：在表面码下编译容错量子启发式算法的子程序并给出资源预算范围，属于“以工作负载驱动的资源与编译权衡分析”。[31]
AutoBraid（MICRO 2021）：聚焦表面码通信/编织路径调度瓶颈，提出框架与并行性分析，体现“编译—体系结构耦合”的 CCF-A 风格。[32]
“Scalable surface-code decoders with parallelization in time”（PRXQuantum 2023）：提出滑动窗口并行解码以消除随规模增长的 backlog 风险，明确“经典处理速度是扩展瓶颈”。[33]
“Parallel window decoding…”（Nature Communications 2023）：并行窗口解码展示可以避免指数级减速，尽管需要延迟部分经典前馈决策（把“延迟”显式纳入权衡）。[34]
Bench-Q（开源）：提供容错资源估算工具，包含图态编译、蒸馏工厂模型、解码器性能模型与体系结构模型等，适合小组做可复现实验。[35]
Lattice Surgery Compiler / 大规模表面码编译（2023–2024）：提出能将任意电路转为晶格手术表面码操作的高性能编译器，并强调可扩展到百万/千万级指令并支持资源估算与流水化。[36]

这些工作共同指向：**容错策略必须与系统性能模型绑定**；而性能模型需要同时考虑量子操作时长、经典处理/通信延迟、以及编译引入的并行结构与魔态需求节奏。[37]

子方向 4.1：百纳秒级反馈与低延迟读出链路

趋势概括：低延迟反馈的衡量指标逐步从“能反馈”走向“可量化的端到端延迟预算”，并服务于动态电路与 QEC。

代表性结果：

FPGA 反馈触发延迟：早期工作报告可产生相对模拟输入信号 **110 ns** 的反馈触发延迟，用于实时判别与反馈操作（对百纳秒级反馈的经典标杆）。[4]
动态超导量子计算读出电子学：报告读出链在 FPGA 上实现从最后采样输入到“读出有效输出”的 **40 ns** 延迟，并给出跨仪器反馈链路通信延迟指标，属于更工程化的低延迟读出实现。[5]

子方向 4.2：可扩展测控系统的软硬件协同栈与开源平台

小组设备不足时，这个子方向的策略是：**以开源控制栈为“目标体系结构”做协同设计/编译研究**，必要时用少量开发板或仿真替代真实量子芯片。

代表性平台：

QubiC（2020 arXiv / IEEE TQE）：开源 FPGA 控制与测量系统，包含室温电子硬件、FPGA gateware 与工程软件，强调模块化与全栈可访问性，并在超导处理器上演示表征与基准实验。[38]
QubiC 2.0（2023）：进一步强调 mid-circuit measurement 与 feed-forward 能力，并报告多 FPGA 同步与条件电路的前馈实验。[39]
QICK（2021/2022）：基于 RFSoC（如 ZCU111 等）的一体化开源控制器，支持高达 GHz 载频的直接合成控制脉冲，并报告其数字延迟等对 QEC/反馈的重要指标。[40]

子方向 4.3：语言/运行时对实时测控与动态电路的支撑

关键脉络是“语言/IR 显式表达实时控制 → 控制系统与编译工具链重构 → 动态电路/实时反馈成为可用能力”。

OpenQASM 3：在语言层提供控制流、时序、脉冲与校准块，强调并行与数据流模型需要贴近控制硬件。 [23]
IBM 动态电路实践：IBM 指出为了支持 feed-forward/feedback 控制流，需要更新 OpenQASM 并重构编译工具链；并在 2022 年将动态电路部署到多台“快读出”系统上。 [16]
动态电路算法示范：相关论文指出当系统噪声与延迟足够低时，动态电路的实时执行可带来可见优势。 [41]
QIR 与 QAT：QIR 作为 LLVM-based 的量子 IR 规范与生态工具链，为混合量子—经典程序提供统一接口；QAT 用于将通用 QIR 变换为满足特定后端要求的 QIR 并进行合规性验证。 [42]

方法与平台对比表

编译验证与噪声/漂移自适应方法对比（覆盖 3.1–3.3）

方向/方法	核心假设（抽象层）	代表性评估规模/场景	关键指标（鲁棒/性能）	开源/可复现性	是否需专用硬件
QWIRE Practice（Coq 形式化电路）	电路语义可用密度矩阵超算子形式化；适合证明程序性质	以小型量子程序与证明为主 [18]	正确性证明（语义性质）	有 Coq 实现与论文 [43]	否
VOQC/SQIR（Coq 验证优化器）	低层 IR（SQIR）可定义矩阵语义并对变换证明语义保持 [19]	对一组电路基准验证优化效果（论文报告与工业编译器可比） [44]	语义保持证明 + 优化收益	论文与代码公开 [45]	否
Proving Quantum Programs Correct（ITP 2021）	SQIR 既可作 IR 也可用于“源程序级证明” [46]	验证 Grover、QPE 等算法 [47]	机器检验证明覆盖更复杂算法	有论文与 SQIR 仓库指引 [48]	否
CertiQ（契约 +SMT 验证真实编译 pass）	用注释生成验证条件；将电路等价检查编码为 SMT 约束 [21]	重实现并验证大量 Qiskit pass，发现实现 bug，且通过回归测试 [49]	pass 级正确性；工程适配性	论文公开；可作为设计范式 [49]	否（验证本身）
Noise-Adaptive Compiler Mappings（ASPLOS 2019）	编译应显式利用动态校准数据与噪声参数时变性 [2]	在真实 IBM 公有机环境/测量数据上评估 [2]	成功率提升、编译开销与可扩展性权衡 [2]	论文公开；方法可复刻	不必须（可用云端）
随机化编译（PhysRevX 2021）	系统失配/漂移/串扰导致相干误差；通过随机化将其转为随机噪声 [9]	超导处理器上示范 QFT/随机电路性能提升 [9]	性能提升与可预测性（从实验误差率预测算法表现） [50]	论文公开；可用仿真复刻部分结论	不必须（云端可做小规模验证）
动态退耦 DD 编译插入（Qiskit pass）	在已调度电路的 idle 时隙插入 DD 序列以抑制退相干 [10]	作为工程工具与基线（可在仿真/云端测试）	电路保真度/成功率提升、额外深度/时长开销	官方文档可复现 [51]	否（云端可）

方向/方法	核心假设（抽象层）	代表性评估规模/场景	关键指标（鲁棒/性能）	开源/可复现性	是否需专用硬件
Lattice Surgery Compiler / 大规模表面码编译	将逻辑电路翻译为晶格手术/表面码操作，面向大规模 QEC 工作流 [52]	报告可处理百万/千万级指令并支持资源估算/流水化 [53]	编译吞吐、资源估算精度、并行结构	开源项目与文档 [54]	否（主要仿真/编译）
Bench-Q（FT 资源估算）	通过模型化蒸馏工厂/解码器/体系结构估计容错资源	提供图态编译、蒸馏模型、解码模型等工具 [55]	物理资源（qubit-time）、估算吞吐	pip 可装、开源 [35]	否

低延迟测控/反馈系统对比（覆盖 4.1–4.3）

平台/工作	核心假设（体系结构/硬件）	实验规模/对象	延迟/同步等指标	开源性	专用硬件需求
FPGA 低延迟 DSP 反馈（2017/2018 PR Applied）	FPGA 完成实时解调、判决并输出反馈触发	超导比特反馈初始化等实验	报告反馈触发延迟约 **110 ns** [4]	论文公开	需要 FPGA & 射频链（实验级）
动态超导量子计算低延迟读出电子学（AIP Adv 2022）	FPGA 多通道并行处理读出数据，面向动态电路/QEC	读出链路电子学原型	报告读出处理延迟 **40 ns**（从最后采样到输出有效） [5]	论文公开	需要 ADC/FPGA（但可仿真建模）
QubiC（2020）	模块化 FPGA 控制栈+室温电子学+软件，全栈可访问	超导处理器表征、RB 等	强调快速序列加载等能力 [56]	开源栈与站点 [57]	需要 FPGA 开发板（可部分用仿真替代）
QubiC 2.0（2023）	面向 mid-circuit measurement & feed-forward；多 FPGA 同步	bench tests + 条件电路前馈演示	报告多 FPGA 同步与前馈演示 [39]	有论文/实现线索	中等（开发板+同步）
QICK（2021/2022）	RFSoC 一体化多通道控制器，强调数字延迟与开源	超导比特控制/表征	报告将数字延迟作为 QEC/反馈关键指标并给出硬件基准 [58]	开源（含软硬件） [59]	需要 RFSoC（但可作为目标架构建模）

平台/工作	核心假设（体系结构/硬件）	实验规模/对象	延迟/同步等指标	开源性	专用硬件需求
OpenQASM 3 / OpenPulse	语言层显式表达实时控制、时序、脉冲与校准	多硬件生态	支持 timing、defcal、OpenPulse ports/frames/waveforms [60]	规范与实现列表公开 [61]	否
IBM 动态电路（实践）	需要快读出与控制系统重构以支持实时控制流	多系统上线（2022）	指出 mid-circuit measurement/经典操作可能变长，需优化延迟 [12]	文档/博客公开	不必须（用户侧云端）

研究空白与挑战

本节聚焦“哪些研究空白最适合小课题组做出可发表成果”，并给出设备、数据、仿真/验证手段、时间线可行性分析。

面向小课题组的结构性空白

空白一：实时量子指令（OpenQASM 3 / OpenPulse / 动态电路）缺少“可机检的统一语义+可验证编译链”
OpenQASM 3 规范强调 timing、控制流与 OpenPulse 校准语法 [60]，但学界已验证的编译框架多集中在“门级电路 IR”（SQIR、QWIRE）或“编译 pass 等价”的层面（VOQC、CertiQ）。[62] 将验证推进到“时序/反馈/寄存器/校准”的统一语义层，是明显缺口，同时**不依赖昂贵硬件**（主要依赖形式化与测试基准）。

空白二：漂移感知编译缺少“可复现实验协议”：如何用公开校准数据做统计显著的漂移建模与闭环评测
Noise-adaptive mapping 清晰指出硬件参数在时间与空间上有细粒度变化，且可用于提升成功率 [2]；随机化编译与脉冲鲁棒优化工作则更直接讨论漂移/控制不稳定性导致的误差并提供实验结果 [63]。但学界常见问题是：很多论文难以复现“漂移过程”，因为漂移是时间序列、与具体硬件有关。小组可以贡献 **“可复现漂移基准（benchmark）+ 编译策略评测协议”**：收集云端校准数据、定义漂移强度/相关时间、与编译策略的稳健性指标。

空白三：QEC/动态电路的系统性能权衡缺少“端到端延迟预算模型”与“编译可调 knobs”映射
动态电路文档明确指出 mid-circuit measurement 与经典操作时间可能抵消深度降低带来的收益，降低这些延迟是重点方向 [64]；解码研究则强调 classic processing speed 决定是否出现 backlog [65]。然而从“编译器输出的结构（并行度、测量频率、反馈点）”到“控制系统/解码器延迟预算”的映射仍缺少统一模型。小组可以用仿真器（Stim、PyMatching）构建端到端模型并做灵敏度分析。[15]

设备、数据与验证手段可行性

设备方面：真正的百纳秒反馈测控需要 FPGA+高速 ADC/DAC+射频链，成本与工程复杂度较高；但你们可以采用“软优先”路线：把 QICK、QubiC 作为目标架构参考（开源设计与论文提供关键延迟/功能需求），在仿真中建模延迟与吞吐，再把硬件验证压缩到“云端动态电路/小规模控制实验”或“购买 1 块 RFSoC 开发板做纯数字链路验证”。[66]

数据方面：编译与漂移研究不必依赖私有数据。Noise-adaptive mapping 已表明可以基于真实系统测量/校准数据评估编译策略 [2]；动态电路与 OpenQASM 3 工具链也公开可用 [67]。

验证手段方面：
形式化验证：QWIRE/SQIR/VOQC、CertiQ、VyZX 提供可复用框架与方法学 [68]。
仿真与 QEC：Stim（高性能稳定子仿真）、PyMatching（MWPM 解码器）为延迟/解码/容错实验提供强基础设施 [69]。
容错资源估算：Bench-Q、Lattice Surgery Compiler 提供编译与估算链条，适合系统权衡研究 [70]。

时间线：18 个月内形成 CCF-A 主稿的可行路径通常是“**先做一个完整、可复现的研究闭环**（基准+方法+评估）”，再争取硬件验证作为加分项而非前置条件。

可行研究路线图

本节对三个推荐课题分别给出：目标、创新点、方法论、实验/仿真方案（含软硬件清单与替代）、评估指标、风险与应对，并用 18–36 个月甘特图给出里程碑。

课题 A：OpenQASM 3 / OpenPulse 的时序—反馈形式化语义与可验证编译

研究目标

构建一条“从实时量子程序到可执行控制序列”的 **可验证编译链**，覆盖至少以下要素：
OpenQASM 3 控制流与时序（durations、delays、boxing/对齐意图等）[71]、OpenPulse 的 ports/frames/waveforms 与 play/capture 指令 [24]、以及动态电路中的测量—经典条件分支（if）语义。[12]

可发表的创新点设计

创新点一：**“时序/反馈可验证”的 IR 分层**
借鉴 SQIR“为证明而设计”的经验 [72]，提出一个包含时间戳、资源（控制端口/寄存器）与反馈边的 IR（可称 RT-SQIR），并给出从 OpenQASM 3 子集到 RT-SQIR 的翻译及证明。

创新点二：**把“编译正确性”从等价扩展为“时序约束满足 + 反馈一致性 + 安全属性”**
例如：无资源冲突（同一 port 同时 play）、无读写越界（fast register vs memory）、满足对齐与延迟意图、反馈分支在给定延迟模型下可在相干时间窗内完成等。OpenQASM 3 强调时序解析依赖后端上下文，而语言层提供高层设计意图 [73]；这是形式化研究的天然切入点。

创新点三：**“脉冲级语义的可计算近似界”**（利用你的哈密顿模拟背景）
在不追求完整物理精确的前提下，为 piecewise-constant/limited-bandwidth 波形给出一个可证明的近似语义（例如用 Trotter 或 Magnus 展开误差界），使“门级目标”与“脉冲级实现”之间存在可证明关系——这直接回应 Qiskit Pulse 论文强调的“需要脉冲级指令以精准控制硬件”。[25]

方法论

形式化框架建议两条路线择其一（小组资源有限时建议“先小后大”）：

路线 A1（Coq 生态）：以 QWIRE/SQIR/VOQC 为基础扩展时间与反馈语义。QWIRE 与 SQIR 均有 Coq 实现与论文资源，适合快速进入可证明开发。[74]
路线 A2（契约+SMT 生态）：借鉴 CertiQ 的“注释→验证条件→SMT”路线，把 OpenQASM 3 子集的编译 pass 表述为带契约的变换，并生成可自动化验证条件。[21]

实验/验证方案与可复制步骤

软件清单（最低配）

1) OpenQASM 3 parser + Qiskit 导入器
- 安装：

```
python -m venv venv && source venv/bin/activate
pip install qiskit qiskit_qasm3_import
```

qiskit_qasm3_import 的安装方式在 PyPI/IBM 文档中给出（也可通过 `qiskit[qasm3-import]` 安装）。 [75]

2) 形式化验证工具
- Coq（配合 QWIRE/SQIR/VOQC）：从 QWIRE/SQIR 仓库入手 [76]
- 或 SMT（Z3 等）配合 CertiQ 风格工具链：以 CertiQ 论文中的方法学为蓝本 [49]

3) 参考规范
OpenQASM Live Spec（包含 timing、OpenPulse Grammar、校准块等章节） [60]

可复现的核心实验

实验 A：语义一致性回归套件
- 收集一组含 timing/defcal/openpulse 的 OpenQASM 3 测例（可从 openqasm 仓库示例扩展） [77]
- 编译到 RT-SQIR
- 在“抽象语义解释器”与“参考执行（例如 Qiskit/模拟器或你自建的 pulse-level toy simulator）”之间做等价/包含关系检查

实验 B：安全属性验证
- 证明编译后程序满足：port 冲突自由、寄存器访问合法、对齐/延迟意图满足（以 OpenQASM timing 章节为规范来源） [78]

硬件需求与替代

不需要自有量子测控硬件；如要补强论文说服力，可选“云端运行一小部分 dynamic circuits / OpenQASM 3 程序”作为演示，IBM 动态电路文档与指南提供接口说明。 [12]

评估指标

正确性：证明覆盖率（语言特性子集覆盖）、自动化程度（证明脚本规模/自动化比例）、bug-finding 能力（类似 CertiQ 发现 bug 的价值主张）。 [49]
工程性能：编译时间、生成代码长度、与现有工具链（Qiskit 导入器、OpenQASM 规范）互操作性。 [79]

风险与应对

风险：OpenQASM 3 / OpenPulse 规范演进与实现差异可能导致目标移动。OpenQASM 规范是 “live specification”，存在版本迭代。 [80]
应对：明确“受控子集 + 可扩展语义模块化”；把规范版本、特性子集写入 artifact 与论文，变更通过 regression suite 捕捉。

课题 B：漂移感知自适应编译 + 随机化编译/动态退耦联合优化

研究目标

构建一个端到端可复现框架：
(1) 收集并建模云端后端校准数据的时间序列漂移 → (2) 编译阶段选择映射/调度/误差抑制策略 → (3) 在仿真与少量云端实验中统计评估稳健性。

创新点设计

创新点一：**漂移基准（Drift Benchmark）与可复现实验协议**
现有论文往往一次性报告“某段时间内提升”，但漂移是时间序列。你可定义：漂移强度指标（校准参数变化率/方差）、漂移相关时间、以及编译策略的“稳健区间长度”（类似鲁棒脉冲工作强调校准窗口延长）。[81]

创新点二：**联合优化：映射（noise-adaptive） + 噪声定制（randomized compiling） + idle 抑制（DD）**
Noise-adaptive mapping 为“利用校准数据”提供基线[2]；随机化编译针对失配/漂移/串扰导致的相干误差[50]；Qiskit 的 DD pass 提供工程化可复现实现[10]。三者组合带来清晰的“系统贡献点”：你不仅提出新算法，还提出可复现的联合评估框架。

创新点三：**“编译决策的后验置信度”**
引入统计学习：在给定校准不确定性/漂移模型下，对编译输出的期望成功率给出置信区间，并用极少量探测电路在线更新（这是小组可做但大多论文未系统化的点）。

方法论

数据层：周期性抓取后端校准数据（例如门误差、T1/T2、读出误差等）并形成时间序列，复刻 Noise-adaptive mapping “利用动态校准”的精神。[2]
策略层：
- 映射：改进或复现 noise-adaptive mapping 的 cost function，并加入“漂移风险项”。[2]
- 相干误差处理：加入 randomized compiling 或其简化变体（Pauli twirling 等），以减少漂移导致的不可预测相干误差。[9]
- idle 抑制：对长 idle 区间插入 DD（PadDynamicalDecoupling）。[82]
- 脉冲级扩展（加分）：参考云端鲁棒脉冲优化工作，把“漂移”从门级拓展到脉冲校准（不一定要自己做硬件脉冲实验，仿真也可先做）。[29]

评估层：仿真（噪声模型 + 漂移模型）为主，云端执行为辅。

实验/仿真方案与命令示例

软件清单（建议）

- Qiskit（含 IBM Runtime 组件，若可用）：用于 transpilation、DD pass、以及云端作业。IBM 文档中对动态电路与 classical feedforward 有专门指南。[83]
- 可选：tket 开源编译器作为对比。[84]
- 统计/学习：numpy/scipy/pandas + 你熟悉的 ML 框架

最小可复现实验流程

1) 基准电路集：选取 GHZ、QFT 子电路、随机电路/量子体积类电路（尽量小规模以适配云端）
2) 每天/每小时抓取一次校准数据（你可记录为 JSON/CSV），构建漂移时间序列
3) 对同一电路，在不同时间点用不同编译策略编译并评估：

- Baseline：默认 Qiskit transpile
- + Noise-adaptive mapping（复刻或简化 Murali 2019 的噪声加权映射思想） [2]
- + DD 插入（PadDynamicalDecoupling） [82]
- + randomized compiling（若自己实现较重，可先做“门随机化/Pauli twirling”近似版本，再逐步完善到论文级） [50]

DD 插入示意（以 Qiskit 官方 pass 为基线，具体代码按你选的 Qiskit 版本调整）：

```
from qiskit.transpiler.passes import PadDynamicalDecoupling
# 需要在已调度/物理化电路上插入；官方文档解释其工作方式：扫描 delay idle 并插入 DD 序列
```

其“扫描 idle(含 delay)并插入 DD 序列”的定义在 IBM 文档中明确。 [82]

4) 指标：每个时间点统计成功率/保真度 proxy（例如正确输出概率、期望值误差），并报告漂移下的均值、方差、最坏分位数（p95/p99）。

硬件需求与替代

完全不需要自有测控设备；云端硬件只是用于“少量抽样验证趋势”。这与 noise-adaptive mapping 与 randomized compiling 的“在真实系统上测量”的精神一致，但你可以把主贡献放在“可复现协议+联合优化”。 [85]

评估指标

稳健性：在漂移时间窗内的性能衰减率、恢复速度（重新编译/重映射频率）、对漂移强度的敏感度曲线。
成本：编译时间开销、额外门数/深度、额外时序长度（DD 会引入额外脉冲/门，需量化其收益是否抵消开销）。 [86]
可预测性：随机化编译论文强调可从实验误差率预测算法表现的能力，你可用“预测误差 vs 实际误差”作为指标。 [50]

风险与应对

风险：云端硬件接口与动态电路可用性随时间变化；创设“漂移实验”需要长期数据采集。IBM 文档明确动态电路/控制流支持仍在演进，且现实系统中 mid-circuit measurement 时长是关键关注点。 [12]
应对：把核心贡献锚定在“仿真 + 公开校准/统计协议”，云端实验作为补充；并将所有数据与脚本开源，强调可复现。

课题 C：动态电路与 QEC/反馈的端到端延迟预算与编译—系统协同

研究目标

建立一个“从程序结构到延迟预算”的系统模型：输入为含动态控制流的 OpenQASM 3 程序（或抽象动态电路 IR），输出为在给定控制系统/解码器/读出链延迟参数下的可行性判定与性能预测，并进一步提出“编译层可调节 knobs”降低端到端延迟（例如减少反馈点、重排测量、并行化经典计算）。

创新点设计

创新点一：**端到端延迟预算模型（Latency Budget Model）**
把以下延迟显式参数化：

- mid-circuit measurement 与经典 if 的延迟（IBM 指出这些往往比两比特门长，可能抵消深度优势） [64]
- 读出电子学延迟（如 40 ns 级处理链） [5]
- 反馈触发基线（如 110 ns） [87]
- 解码延迟与 backlog 风险（并行化解码研究强调经典处理速度决定可扩展性） [65]
并允许替换为目标架构（QICK/QubiC）参数区间。 [88]

创新点二：**从编译输出结构到延迟的“可解释映射”**
例如：反馈点数量、测量分布密度、逻辑分支复杂度、以及 QEC syndrome 产生速率，如何映射到所需解码吞吐与控制系统时钟（logical clock）。这能直接对接 CCF-A（体系结构/系统）审稿口味。

创新点三：**与 QEC 仿真工具对接的“闭环系统仿真平台”**
用 Stim 生成稳定子电路与错误模型、用 PyMatching 解码并统计逻辑错误率，再把“经典处理延迟”作为约束插入模拟，得到“逻辑保真度 vs 延迟预算 vs 编译结构”的曲线。Stim 与 PyMatching 均强调面向 QEC 的高性能与开源可用性。 [15]

实验/仿真方案与命令示例

软件清单（最低配）

- `stim`（pip 可装） [89]
- `pymatching`（PyMatching） [90]
- 可选：Bench-Q 或 Lattice Surgery Compiler 用于把更高层算法映射到表面码/晶格手术并获取资源节奏信息 [70]

安装示例：

```
pip install stim pymatching benchq
```

其中 Stim 与 Bench-Q 的 pip 安装路径在其官方仓库/PyPI 页面给出。 [91]

核心仿真实验设计

实验 C1：固定 QEC 码距 d，扫描“经典处理延迟/吞吐”
- 用 Stim 构造表面码周期测量电路（或用现成示例/生成器）
- 通过 PyMatching 解码得到逻辑错误率
- 引入“每轮解码可用时间 Tdecode_budget”，当解码时间超过预算则产生 backlog（可用排队模型近似）
- 输出：阈值曲线（backlog 临界点）、逻辑错误率随延迟变化曲线

实验 C2：编译结构对延迟预算的影响
- 将动态电路/反馈点数量作为变量，构造不同控制流密度的电路（OpenQASM 3 表达或抽象 IR） [11]
- 在模型中计算：测量频率↑ → 经典处理压力↑；反馈点↑ → mid-circuit measurement 延迟关键性↑
- 用 IBM 文档的“mid-circuit measurement 时间可能抵消收益”作为现实动机 [64]

硬件需求与替代

无需硬件；如果想加分，可引用公开硬件实现作为参数实例：例如 40 ns 读出处理、110 ns 反馈触发的学术标杆 [92]，或 QICK/QubiC 的系统能力作为目标架构场景。 [88]

评估指标

系统可行性：在给定延迟预算下可支持的最大码距/反馈频率/动态控制流复杂度。
性能：逻辑错误率、吞吐（每秒可执行的 QEC 轮数/动态电路 shot 数）、以及 backlog 概率。
协同收益：在相同硬件参数下，你的编译优化能把可行工作负载边界推到哪里（例如允许更多反馈点或更大 d）。

风险与应对

风险：审稿人可能要求“硬件实验”。
应对：把论文定位在“系统建模+仿真+可复现开源工具链”，并用公开硬件结果作参数校准；如果条件允许，补一个“云端动态电路小实验”做 sanity check。动态电路已在 IBM 生态实践并强调工具链重构，其系统意义很强。[16]

18–36 个月里程碑甘特图（适用于三课题并行/择一主攻）

```
gantt
    title 量子编译验证与低延迟协同研究路线（18–36个月）
    dateFormat  YYYY-MM-DD
    axisFormat  %Y-%m

    section 共同基础设施（0–6个月）
    文献与基线复现（Qiskit/OpenQASM3/SQIR/CertiQ/Stim）       :a1, 2026-03-15, 90d
    统一基准集与数据管线（漂移日志/电路集/评测脚本）            :a2, 2026-04-15, 120d
    开源仓库与可复现实验CI（artifact规范化）                    :a3, 2026-05-15, 120d

    section 课题A：可验证实时指令编译（3–18个月）
    定义RT语义子集与IR（timing+feedback+openpulse子集）         :b1, 2026-06-01, 120d
    翻译器原型（OpenQASM3->RT-IR->后端）                        :b2, 2026-08-01, 150d
    关键性质证明（资源冲突/时序约束/反馈一致性）                  :b3, 2026-10-01, 180d
    论文产出：POPL/PLDI/ICSE方向主稿                             :b4, 2027-03-01, 120d
    扩展：脉冲近似语义与误差界（可选，18–36个月）                  :b5, 2027-07-01, 240d

    section 课题B：漂移感知自适应编译（3–18个月）
    漂移数据采集与建模（统计指标/相关时间/漂移强度）             :c1, 2026-06-01, 150d
    联合优化器（映射+RC/DD组合策略）                              :c2, 2026-09-01, 180d
    仿真+少量云端验证（统计显著性与消融）                          :c3, 2026-12-01, 150d
    论文产出：ASPLOS/MICRO/ISCA方向主稿                           :c4, 2027-05-01, 120d
    扩展：脉冲鲁棒/闭环校准（可选，18–36个月）                     :c5, 2027-08-01, 240d

    section 课题C：端到端延迟预算与协同（3–24个月）
    延迟预算模型（measurement/feedback/decoder/readout）          :d1, 2026-06-15, 150d
    QEC+反馈仿真平台（Stim+PyMatching+排队/吞吐模型）             :d2, 2026-09-15, 180d
    编译knobs设计与系统灵敏度分析（可解释结论）                    :d3, 2027-02-01, 150d
    论文产出：MICRO/ASPLOS/OSDI系统论文                           :d4, 2027-07-01, 120d
    扩展：与开源控制栈参数对标（QICK/QubiC）                       :d5, 2027-10-01, 180d
```

论文写作与投稿策略

本节为三个推荐课题分别给出：可能的 CCF-A 目标、拟投稿题目、核心贡献点、基线与对比清单、预计结果类型。

课题 A 投稿策略

目标会议/期刊（倾向 CCF-A）：
POPL / PLDI（编程语言与编译 + 形式化验证）、ICSE（软件工程与可靠性），或（若更偏体系结构语义）ASPLOS。你可以用 CertiQ（真实编译器验证）[49] 与 VOQC/SQIR（Coq 验证 IR/优化）[44] 作为相关工作锚点，并把差异写成“面向实时控制与反馈的新验证维度”。

拟投稿题目示例：
- 《Verified Real-time Quantum Control Compilation: A Formal Semantics and Certified Transpilation for OpenQASM 3 and OpenPulse》
- 《Timing- and Feedback-Safe Quantum Compilation: Mechanized Proofs for Dynamic Circuits》

核心贡献点（写作时建议用“三段式”叙述）：
1) 给出 OpenQASM 3 + OpenPulse 子集的形式化语义（覆盖 timing 与反馈）。[60]
2) 实现并证明一条编译链的正确性（语义保持/约束满足）。参考 VOQC、CertiQ 的证明/SMT方法学。[62]
3) 以回归套件展示可发现实现/规范不一致问题，或证明强安全属性（资源冲突自由、寄存器安全、反馈一致性）。

基线对比清单：
- 只做电路等价的验证框架：VOQC/SQIR [44]
- 工程化 pass 验证框架：CertiQ [49]
- 语言/IR 互操作：OpenQASM 3 规范与 qiskit_qasm3_import 工具链 [93]

预计结果类型：以**理论证明 + 原型实现 + 回归/bug finding 实验**为主；硬件验证作为可选加分。

课题 B 投稿策略

目标会议/期刊（倾向 CCF-A）：
ASPLOS / MICRO / ISCA（编译—体系结构协同、噪声感知系统），也可考虑 NeurIPS（若核心方法是学习/优化算法且贡献扎实，但需谨慎定位）。主线参考 ASPLOS 2019 noise-adaptive mapping [2] 与 randomized compiling（PhysRevX 2021）[94]。

拟投稿题目示例：
- 《DriftBench: Reproducible Drift Modeling and Adaptive Compilation Using Public Calibration Telemetry》
- 《Joint Noise Tailoring and Drift-Aware Mapping for NISQ Programs》

核心贡献点：
1) 给出漂移建模与评测协议（可复现数据集/脚本）。
2) 提出联合优化策略：噪声感知映射 + 随机化编译 + DD 插入，并做消融。[95]
3) 证明“稳健性”而非仅“平均提升”：报告最坏分位数、漂移窗口长度等。

基线对比清单：
- Qiskit 默认 transpile
- Murali 2019 noise-adaptive mapping（复刻其代价函数/策略）[2]

- t|ket〉编译器作为第三方 baseline [28]
- 仅 DD、仅 randomized compiling、以及联合策略。 [96]

预计结果类型：**大规模仿真统计 + 少量云端硬件验证**（强调可复现与统计显著性）。

课题 C 投稿策略

目标会议/期刊（倾向 CCF-A）：
ASPLOS / MICRO（系统与编译协同最合适），如果系统层更强（例如提出新的运行时/调度系统）也可尝试 OSDI/SOSP，但需要更重的系统实现与更强的普适性叙事。

拟投稿题目示例：
- 《From Programs to Nanoseconds: End-to-End Latency Budgeting for Dynamic Circuits and Quantum Error Correction》
- 《Compiler-Exposed Knobs for Real-Time Quantum Feedback: A Simulation-Backed Co-Design Study》

核心贡献点：
1) 端到端延迟预算模型，明确 mid-circuit measurement 与经典控制流延迟的关键性（IBM 文档明确这是改进重点）。 [64]
2) 用开源 QEC 仿真与解码工具构建闭环平台（Stim/PyMatching）。 [15]
3) 提供可解释的系统结论与设计建议（例如在哪些延迟区间必须硬件加速，在哪些区间编译优化足够）。

基线与对比清单：
- 不考虑延迟预算的纯逻辑错误率评估
- 只考虑固定解码延迟 vs 引入 backlog 模型
- 采用不同公开延迟参数实例（40 ns readout、110 ns feedback 等） [92]
- 可选：将参数对标开源控制栈能力（QICK/QubiC）。 [88]

预计结果类型：**系统建模 + 大规模仿真 + 参数对标**；若能争取合作做少量硬件验证，可显著加分（但不作为主依赖）。

关键参考文献与优先级

按“优先官方/原始论文 → 主要会议/期刊 → 开源项目/官方文档 → 厂商白皮书/博客（仅作工程背景）”排序。

高优先级（必须精读/复现）

OpenQASM 3 论文与规范： [97]
OpenPulse Grammar（OpenQASM 生态中的校准语法）： [24]
Qiskit Pulse 论文： [25]
VOQC / SQIR（验证优化器与 IR）： [98]
CertiQ（真实编译器 pass 验证）： [21]
Noise-Adaptive Compiler Mappings（ASPLOS 2019）： [27]
Randomized Compiling（PhysRevX 2021）： [3]
动态电路与实时执行：IBM 指南/博客 + PRL 工作： [99]
低延迟反馈/DSP 与读出电子学： [100]
QubiC / QubiC 2.0 / QICK（开源控制栈）： [101]
Stim / PyMatching（QEC 仿真与解码）： [102]

Bench-Q 与 Lattice Surgery Compiler（容错资源估算与编译）： 70
QIR 规范与生态（互操作 IR）： 26

中优先级（扩展阅读/对比基线）

t|ket〉编译器论文与开源仓库： 28
AutoBraid（MICRO 2021，FT 通信/并行调度）： 32
并行解码与背压（PRXQuantum 2023、Nature Communications 2023）： 65
VyZX（图形化量子语言验证）： 22
Qiskit 动态退耦编译 pass 文档： 51
Q-CTRL 脉冲鲁棒优化论文（用于“漂移/控制不稳定”动机与指标）： 29

低优先级（工程背景/中文辅助材料，选择性引用）

CAS 等对 IBM 路线图与动态电路的中文摘要（用于背景叙述，不作为核心证据）： 103

---

1 7 11 23 97 https://arxiv.org/abs/2104.14722
https://arxiv.org/abs/2104.14722

2 27 85 95 https://arxiv.org/abs/1901.11054
https://arxiv.org/abs/1901.11054

3 94 https://link.aps.org/doi/10.1103/PhysRevX.11.041039
https://link.aps.org/doi/10.1103/PhysRevX.11.041039

4 87 100 https://arxiv.org/abs/1709.01030
https://arxiv.org/abs/1709.01030

5 92 https://pubs.aip.org/aip/adv/article/12/4/045024/2818923/Low-latency-readout-electronics-for-dynamic
https://pubs.aip.org/aip/adv/article/12/4/045024/2818923/Low-latency-readout-electronics-for-dynamic

6 40 59 https://arxiv.org/abs/2110.00557
https://arxiv.org/abs/2110.00557

8 17 18 68 74 https://arxiv.org/abs/1803.00699
https://arxiv.org/abs/1803.00699

9 50 63 https://arxiv.org/abs/2010.00215
https://arxiv.org/abs/2010.00215

10 82 86 96 https://quantum.cloud.ibm.com/docs/api/qiskit/qiskit.transpiler.passes.PadDynamicalDecoupling
https://quantum.cloud.ibm.com/docs/api/qiskit/qiskit.transpiler.passes.PadDynamicalDecoupling

12 64 83 99 https://quantum.cloud.ibm.com/docs/guides/classical-feedforward-and-control-flow
https://quantum.cloud.ibm.com/docs/guides/classical-feedforward-and-control-flow

13 https://arxiv.org/abs/2406.08491
https://arxiv.org/abs/2406.08491

14 https://arxiv.org/abs/2410.05202
https://arxiv.org/abs/2410.05202

15 69 89 91 https://github.com/quantumlib/stim
https://github.com/quantumlib/stim

16 https://www.ibm.com/quantum/blog/quantum-dynamic-circuits
https://www.ibm.com/quantum/blog/quantum-dynamic-circuits

19 45 98 https://arxiv.org/abs/1912.02250
https://arxiv.org/abs/1912.02250

20 47 https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2021.21
https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2021.21

21 49 https://arxiv.org/abs/1908.08963
https://arxiv.org/abs/1908.08963

22 https://arxiv.org/abs/2311.11571
https://arxiv.org/abs/2311.11571

24 https://openqasm.com/language/openpulse.html
https://openqasm.com/language/openpulse.html

25 https://arxiv.org/abs/2004.06755
https://arxiv.org/abs/2004.06755

26 42 https://github.com/qir-alliance/qir-spec/blob/main/specification/README.md
https://github.com/qir-alliance/qir-spec/blob/main/specification/README.md

28 https://arxiv.org/abs/2003.10611
https://arxiv.org/abs/2003.10611

29 81 https://arxiv.org/abs/2010.08057
https://arxiv.org/abs/2010.08057

30 https://quantum-journal.org/papers/q-2019-03-05-128/
https://quantum-journal.org/papers/q-2019-03-05-128/

31 https://link.aps.org/doi/10.1103/PRXQuantum.1.020312
https://link.aps.org/doi/10.1103/PRXQuantum.1.020312

32 https://people.cs.rutgers.edu/zz124/assets/pdf/micro21.pdf
https://people.cs.rutgers.edu/zz124/assets/pdf/micro21.pdf

33 37 65 https://link.aps.org/doi/10.1103/PRXQuantum.4.040344
https://link.aps.org/doi/10.1103/PRXQuantum.4.040344

34 https://www.nature.com/articles/s41467-023-42482-1.pdf
https://www.nature.com/articles/s41467-023-42482-1.pdf

35 55 70 https://github.com/zapatacomputing/benchq
https://github.com/zapatacomputing/benchq

36 53 https://arxiv.org/abs/2302.02459
https://arxiv.org/abs/2302.02459

38 56 101 https://arxiv.org/abs/2101.00071
https://arxiv.org/abs/2101.00071

39 https://arxiv.org/abs/2309.10333
https://arxiv.org/abs/2309.10333

41 https://arxiv.org/pdf/2102.01682
https://arxiv.org/pdf/2102.01682

43 76 https://github.com/inQWIRE/QWIRE
https://github.com/inQWIRE/QWIRE

44 62 https://rand.cs.uchicago.edu/files/popl_2021_full.pdf
https://rand.cs.uchicago.edu/files/popl_2021_full.pdf

46 72 https://drops.dagstuhl.de/storage/00lipics/lipics-vol193-itp2021/LIPIcs.ITP.2021.21/LIPIcs.ITP.2021.21.pdf
https://drops.dagstuhl.de/storage/00lipics/lipics-vol193-itp2021/LIPIcs.ITP.2021.21/LIPIcs.ITP.2021.21.pdf

48 https://github.com/inQWIRE/SQIR
https://github.com/inQWIRE/SQIR

51 https://qiskit.qotlabs.org/docs/guides/dynamical-decoupling-pass-manager
https://qiskit.qotlabs.org/docs/guides/dynamical-decoupling-pass-manager

52 https://quantum-journal.org/papers/q-2024-05-22-1354/
https://quantum-journal.org/papers/q-2024-05-22-1354/

54 https://lattice-surgery-compiler.readthedocs.io/en/latest/
https://lattice-surgery-compiler.readthedocs.io/en/latest/

57 https://lbl-qubic.gitlab.io/
https://lbl-qubic.gitlab.io/

58 66 88 https://ar5iv.labs.arxiv.org/html/2110.00557
https://ar5iv.labs.arxiv.org/html/2110.00557

60 71 https://openqasm.com/
https://openqasm.com/

61 77 80 93 https://github.com/openqasm/openqasm
https://github.com/openqasm/openqasm

67 79 https://github.com/Qiskit/qiskit-qasm3-import
https://github.com/Qiskit/qiskit-qasm3-import

73 78 https://openqasm.com/language/delays.html
https://openqasm.com/language/delays.html

75 https://pypi.org/project/qiskit-qasm3-import/
https://pypi.org/project/qiskit-qasm3-import/

84 https://github.com/Quantinuum/tket
https://github.com/Quantinuum/tket

90 https://arxiv.org/abs/2105.13082
https://arxiv.org/abs/2105.13082

102 https://arxiv.org/abs/2103.02202
https://arxiv.org/abs/2103.02202

103 https://casisd.cas.cn/zkcg/ydkb/kjqykb/2022/202207/202209/t20220927_6517713.html
https://casisd.cas.cn/zkcg/ydkb/kjqykb/2022/202207/202209/t20220927_6517713.html